

CSVInspect

2025-02-27

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
vehicleData = read.csv("vehicles.csv")
emissionsData = read.csv("1970-2022StateCO2Emissions.csv")
```

```
head(vehicleData)
```

```
##   barrels08 barrelsA08 charge120 charge240 city08 city08U cityA08 cityA08U
## 1  14.16714          0          0          0    19      0      0      0
## 2  27.04636          0          0          0     9      0      0      0
## 3  11.01889          0          0          0    23      0      0      0
## 4  27.04636          0          0          0    10      0      0      0
## 5  15.65842          0          0          0    17      0      0      0
## 6  13.52318          0          0          0    21      0      0      0
##   cityCD cityE cityUF co2 co2A co2TailpipeAGpm co2TailpipeGpm comb08 comb08U
## 1      0      0      0 -1  -1          0      423.1905      21      0
## 2      0      0      0 -1  -1          0      807.9091      11      0
## 3      0      0      0 -1  -1          0      329.1481      27      0
## 4      0      0      0 -1  -1          0      807.9091      11      0
## 5      0      0      0 -1  -1          0      467.7368      19      0
## 6      0      0      0 -1  -1          0      403.9545      22      0
##   combA08 combA08U combE combinedCD combinedUF cylinders displ
## 1      0      0      0          0          0          4      2.0
## 2      0      0      0          0          0          12      4.9
## 3      0      0      0          0          0          4      2.2
## 4      0      0      0          0          0          8      5.2
## 5      0      0      0          0          0          4      2.2
## 6      0      0      0          0          0          4      1.8
##           drive engId   eng_dscr feScore fuelCost08 fuelCostA08
## 1      Rear-Wheel Drive  9011    (FFS)      -1      2150          0
## 2      Rear-Wheel Drive 22020  (GUZZLER)    -1      4100          0
## 3      Front-Wheel Drive 2100    (FFS)      -1      1650          0
```

```

## 4          Rear-Wheel Drive 2850          -1          4100          0
## 5 4-Wheel or All-Wheel Drive 66031 (FFS,TRBO) -1          3100          0
## 6          Front-Wheel Drive 66020          (FFS) -1          2050          0
## fuelType          fuelType1 ghgScore ghgScoreA highway08 highway08U highwayA08
## 1 Regular Regular Gasoline -1          -1          25          0          0
## 2 Regular Regular Gasoline -1          -1          14          0          0
## 3 Regular Regular Gasoline -1          -1          33          0          0
## 4 Regular Regular Gasoline -1          -1          12          0          0
## 5 Premium Premium Gasoline -1          -1          23          0          0
## 6 Regular Regular Gasoline -1          -1          24          0          0
## highwayA08U highwayCD highwayE highwayUF hlv hpv id lv2 lv4 make
## 1          0          0          0          0 0 0 1 0 0 Alfa Romeo
## 2          0          0          0          0 0 0 10 0 0 Ferrari
## 3          0          0          0          0 19 77 100 0 0 Dodge
## 4          0          0          0          0 0 0 1000 0 0 Dodge
## 5          0          0          0          0 0 0 10000 0 14 Subaru
## 6          0          0          0          0 0 0 10001 0 15 Subaru
## model mpgData phevBlended pv2 pv4 range rangeCity rangeCityA
## 1 Spider Veloce 2000 Y false 0 0 0 0 0
## 2 Testarossa N false 0 0 0 0 0
## 3 Charger Y false 0 0 0 0 0
## 4 B150/B250 Wagon 2WD N false 0 0 0 0 0
## 5 Legacy AWD Turbo N false 0 90 0 0 0
## 6 Loyale N false 0 88 0 0 0
## rangeHwy rangeHwyA trany UCity UCityA UHighway UHighwayA
## 1          0          0 Manual 5-spd 23.3333 0 35.0000 0
## 2          0          0 Manual 5-spd 11.0000 0 19.0000 0
## 3          0          0 Manual 5-spd 29.0000 0 47.0000 0
## 4          0          0 Automatic 3-spd 12.2222 0 16.6667 0
## 5          0          0 Manual 5-spd 21.0000 0 32.0000 0
## 6          0          0 Automatic 3-spd 27.0000 0 33.0000 0
## VClass year youSaveSpend baseModel guzzler trans_dscr tCharger
## 1 Two Seaters 1985 -2750 Spider NA
## 2 Two Seaters 1985 -12500 Testarossa T NA
## 3 Subcompact Cars 1985 -250 Charger SIL NA
## 4 Vans 1985 -12500 B150/B250 Wagon NA
## 5 Compact Cars 1993 -7500 Legacy/Outback TRUE
## 6 Compact Cars 1993 -2250 Loyale NA
## sCharger atvType fuelType2 rangeA evMotor mfrCode c240Dscr charge240b
## 1 0
## 2 0
## 3 0
## 4 0
## 5 0
## 6 0
## c240bDscr createdOn modifiedOn startStop
## 1 Tue Jan 01 00:00:00 EST 2013 Tue Jan 01 00:00:00 EST 2013
## 2 Tue Jan 01 00:00:00 EST 2013 Tue Jan 01 00:00:00 EST 2013
## 3 Tue Jan 01 00:00:00 EST 2013 Tue Jan 01 00:00:00 EST 2013
## 4 Tue Jan 01 00:00:00 EST 2013 Tue Jan 01 00:00:00 EST 2013
## 5 Tue Jan 01 00:00:00 EST 2013 Tue Jan 01 00:00:00 EST 2013
## 6 Tue Jan 01 00:00:00 EST 2013 Tue Jan 01 00:00:00 EST 2013
## phevCity phevHwy phevComb
## 1 0 0 0

```

```
## 2      0      0      0
## 3      0      0      0
## 4      0      0      0
## 5      0      0      0
## 6      0      0      0
```

```
head(emissionsData)
```

```
##      State X1970 X1971 X1972 X1973 X1974 X1975 X1976 X1977 X1978 X1979 X1980
## 1  Alabama 102.7  98.5 105.0 109.6 108.8 107.8 108.1 111.7 106.7 111.6 107.2
## 2  Alaska  11.4  12.6  13.4  12.5  12.8  14.5  16.0  18.0  19.5  17.5  17.4
## 3  Arizona  24.9  27.0  30.3  34.5  36.8  38.2  43.8  50.5  49.3  56.2  52.9
## 4  Arkansas 36.2  35.1  37.2  40.9  39.2  36.4  38.9  41.7  42.5  40.3  37.5
## 5 California 294.7 306.1 313.0 329.5 304.7 311.7 327.1 354.7 345.4 362.1 344.4
## 6  Colorado 43.1  43.6  47.5  51.1  50.5  51.8  55.2  58.3  58.5  58.9  58.4
##      X1981 X1982 X1983 X1984 X1985 X1986 X1987 X1988 X1989 X1990 X1991 X1992 X1993
## 1 103.8  91.0  90.0  95.4 101.8 101.9 104.0 105.2 110.0 109.8 114.1 121.0 125.3
## 2  17.1  24.0  26.0  28.7  29.0  31.4  29.8  30.3  33.3  34.0  34.5  35.9  35.7
## 3  59.9  58.5  54.2  58.5  61.1  56.3  56.4  59.6  65.5  63.1  64.0  66.8  69.1
## 4  43.0  42.8  47.1  45.0  49.2  50.0  47.1  51.0  51.3  50.8  49.8  51.4  50.4
## 5 334.5 299.1 293.8 315.8 320.7 306.7 335.9 344.8 359.5 360.3 349.0 352.8 342.1
## 6  57.4  58.6  56.8  60.5  61.2  60.4  61.0  63.3  64.5  65.8  67.5  68.5  72.0
##      X1994 X1995 X1996 X1997 X1998 X1999 X2000 X2001 X2002 X2003 X2004 X2005 X2006
## 1 123.2 131.0 137.2 134.1 133.5 135.8 142.3 133.3 138.3 139.8 142.0 143.5 145.8
## 2  35.5  40.0  41.1  41.2  42.2  42.8  43.6  42.5  42.7  42.9  46.2  47.5  45.4
## 3  71.8  66.7  68.6  71.7  76.7  80.6  86.6  88.9  88.3  90.5  97.3  97.3 100.5
## 4  54.3  57.7  60.2  59.1  60.5  62.7  63.3  62.5  61.2  62.2  62.5  60.2  62.1
## 5 359.0 348.5 349.5 352.3 362.2 365.6 382.0 385.3 383.7 374.0 392.0 388.8 397.5
## 6  72.6  72.7  75.6  75.8  78.1  80.2  85.3  93.0  91.3  90.7  93.2  95.4  96.3
##      X2007 X2008 X2009 X2010 X2011 X2012 X2013 X2014 X2015 X2016 X2017 X2018 X2019
## 1 147.3 139.4 119.7 132.4 129.4 122.4 120.1 122.3 118.8 113.4 108.1 111.9 105.8
## 2  43.7  39.1  37.2  37.1  37.0  36.1  34.0  33.8  34.9  33.3  33.6  34.4  34.2
## 3 102.3 102.6  93.7  99.4  97.5  95.3  99.0  97.0  94.6  90.4  90.0  93.6  92.1
## 4  63.4  64.2  61.4  66.0  67.3  66.1  68.2  68.7  58.8  61.8  63.8  70.5  64.8
## 5 402.3 383.8 369.8 356.1 341.8 347.9 348.3 344.0 349.7 351.2 354.5 356.7 356.5
## 6  98.9  97.0  92.8  95.5  92.1  90.8  91.7  92.3  90.8  87.9  88.2  89.4  91.1
##      X2020 X2021 X2022 Percent Absolute Percent.1 Absolute.1
## 1  98.0 108.3 109.3    6.4%    6.6    0.9%    1.0
## 2  35.9  39.7  41.3  263.6%   29.9    4.0%    1.6
## 3  79.7  82.6  81.4  226.5%   56.5   -1.5%   -1.2
## 4  54.4  61.7  63.2   74.4%   26.9    2.4%    1.5
## 5 302.0 324.1 326.2   10.7%   31.5    0.7%    2.1
## 6  79.3  84.7  88.7  106.0%   45.6    4.8%    4.0
```

```
# the vehicle data set has a lot of extra information, lets only keep the relevant information
vehicleData2022 = vehicleData %>%
  select(city08U, cityA08U, highway08U, highwayA08U, comb08U, combA08U, co2TailpipeGpm, co2TailpipeAGpm)
  filter(year == 2022 )

colnames(vehicleData2022)[1] = "SFVCityMPG"
colnames(vehicleData2022)[2] = "DFVCityMPG"
colnames(vehicleData2022)[3] = "SFVHighwayMPG"
```

```

colnames(vehicleData2022)[4] = "DFVHighwayMPG"
colnames(vehicleData2022)[5] = "SFVCombinedMPG"
colnames(vehicleData2022)[6] = "DFVCombinedMPG"
colnames(vehicleData2022)[7] = "SFVC02Gpm"
colnames(vehicleData2022)[8] = "DFVC02Gpm"

# lets get the electric vehicles from this data set, which would be a 0 for tailpipe emissions in both
electricVehicles2022 = vehicleData2022[vehicleData2022$fuelType == "Electricity", ]

# get relevant data for only electric vehicles with empty columns that will be filled via the state data
electricVehicles2022 = electricVehicles2022 %>%
  select(make, model, year, fuelType, cityE, highwayE, combE, SFVC02Gpm, SFVCityMPG, SFVHighwayMPG, SFVCombinedMPG)

# the cityE, highwayE, and combE are in kw-hours per 100 miles, but I want to know the effect of 1 kWh
electricVehicles2022 = electricVehicles2022 %>%
  mutate(cityMpkWh = 100 / cityE) %>%
  mutate(highwayMpkWh = 100 / highwayE) %>%
  mutate(combMpkWh = 100 / combE)

# now that that is done, lets make our final table with only the information that we need
electricVehicles2022 = electricVehicles2022 %>%
  select(make, model, year, fuelType, cityMpkWh, highwayMpkWh, combMpkWh, SFVC02Gpm)
colnames(electricVehicles2022)[8] = "gCo2pM"

# now we can go back and refine our data for non electric vehicles
nonElectricVehicleData2022 = vehicleData2022[vehicleData2022$fuelType != "Electricity", ]

# dropping the dual fuel grams of co2 per mile since it is rarely used only 17 times, we will use the v
# also going to remove the hydrogen vehicles as we have not accounted for pollution from hydrogen acquisition
nonElectricVehicleData2022 = nonElectricVehicleData2022 %>%
  select(make, model, year, fuelType, SFVCityMPG, SFVHighwayMPG, SFVCombinedMPG, SFVC02Gpm) %>%
  filter(fuelType != "Hydrogen")

colnames(nonElectricVehicleData2022)[5] = "cityMPG"
colnames(nonElectricVehicleData2022)[6] = "highwayMPG"
colnames(nonElectricVehicleData2022)[7] = "combinedMPG"
colnames(nonElectricVehicleData2022)[8] = "gCo2pM"

# clean and mutate the data to get the information we want

emissionsData$X2022 = as.numeric(as.character(emissionsData$X2022))

## Warning: NAs introduced by coercion

stateData2022 = emissionsData %>%
  select(State, X2022)

# now we need to convert the million tons of CO2 into grams of CO2
stateData2022 = stateData2022 %>%
  mutate(gCo2 = X2022 * 1.1e7) %>%
  mutate(gCo2pkWh = 0)

```

```
head(stateData2022)
```

```
##      State X2022      gCo2 gCo2pkWh
## 1  Alabama 109.3 1202300000      0
## 2  Alaska  41.3  454300000      0
## 3  Arizona 81.4  895400000      0
## 4  Arkansas 63.2  695200000      0
## 5 California 326.2 3588200000      0
## 6  Colorado 88.7  975700000      0
```

```
colnames(stateData2022)[2] = "millionTonsOfCO2"
```

```
head(stateData2022)
```

```
##      State millionTonsOfCO2      gCo2 gCo2pkWh
## 1  Alabama      109.3 1202300000      0
## 2  Alaska       41.3  454300000      0
## 3  Arizona      81.4  895400000      0
## 4  Arkansas     63.2  695200000      0
## 5 California   326.2 3588200000      0
## 6  Colorado     88.7  975700000      0
```

```
# data set form the eia for electricity generation from 2022
```

```
electricityGeneration2022 = read.csv("stateGeneration2022.csv")
```

```
electricityGeneration2022 = electricityGeneration2022 %>%
  mutate(netKWhGeneration = NetGeneration.MWh. * 1000)
```

```
head(electricityGeneration2022)
```

```
##      State AverageRetailPrice.cents.kWh. NetSummerCapacity.MW.
## 1  Alabama      11.59      28910
## 2  Alaska      20.73      2820
## 3  Arizona      11.31      28202
## 4  Arkansas      9.91      14954
## 5 California     22.33      85981
## 6  Colorado     11.75      18092
##      NetGeneration.MWh. TotalRetailSales.MWh. netKWhGeneration
## 1      144788893      87027545      144788893000
## 2       6694128      6002080      6694128000
## 3     104698773      84196517     104698773000
## 4      65905030      48997663      65905030000
## 5     203383857     251869136     203383857000
## 6      58044009      56763041      58044009000
```

```
# lets create some visualizations and split the states into regions
```

```
# the us census bureau divides the US into west, midwest, south and north. So we will follow this divis
westData2022 = stateData2022 %>%
```

```

    filter(State %in% c("Arizona", "Colorado", "Idaho", "Montana", "Nevada",
                        "New Mexico", "Utah", "Wyoming", "Alaska", "California",
                        "Hawaii", "Oregon", "Washington"))

midWestData2022 = stateData2022 %>%
  filter(State %in% c("Illinois", "Indiana", "Michigan", "Ohio", "Wisconsin", "Iowa", "Kansas", "Minnesota",
                      "Nebraska", "North Dakota", "South Dakota"))

southData2022 = stateData2022 %>%
  filter(State %in% c("Delaware", "Florida", "Georgia", "Maryland", "North Carolina", "South Carolina",
                      "West Virginia", "Alabama", "Kentucky", "Mississippi", "Tennessee", "Arkansas", "Louisiana",
                      "Oklahoma", "Texas", "District of Columbia"))

northData2022 = stateData2022 %>%
  filter(State %in% c("Connecticut", "Maine", "Massachusetts", "New Hampshire", "Rhode Island", "Vermont", "New Jersey",
                      "New York", "Pennsylvania"))

# to ensure all data was correctly divided
sum(length(westData2022$State)) + sum(length(midWestData2022$State)) + sum(length(southData2022$State))

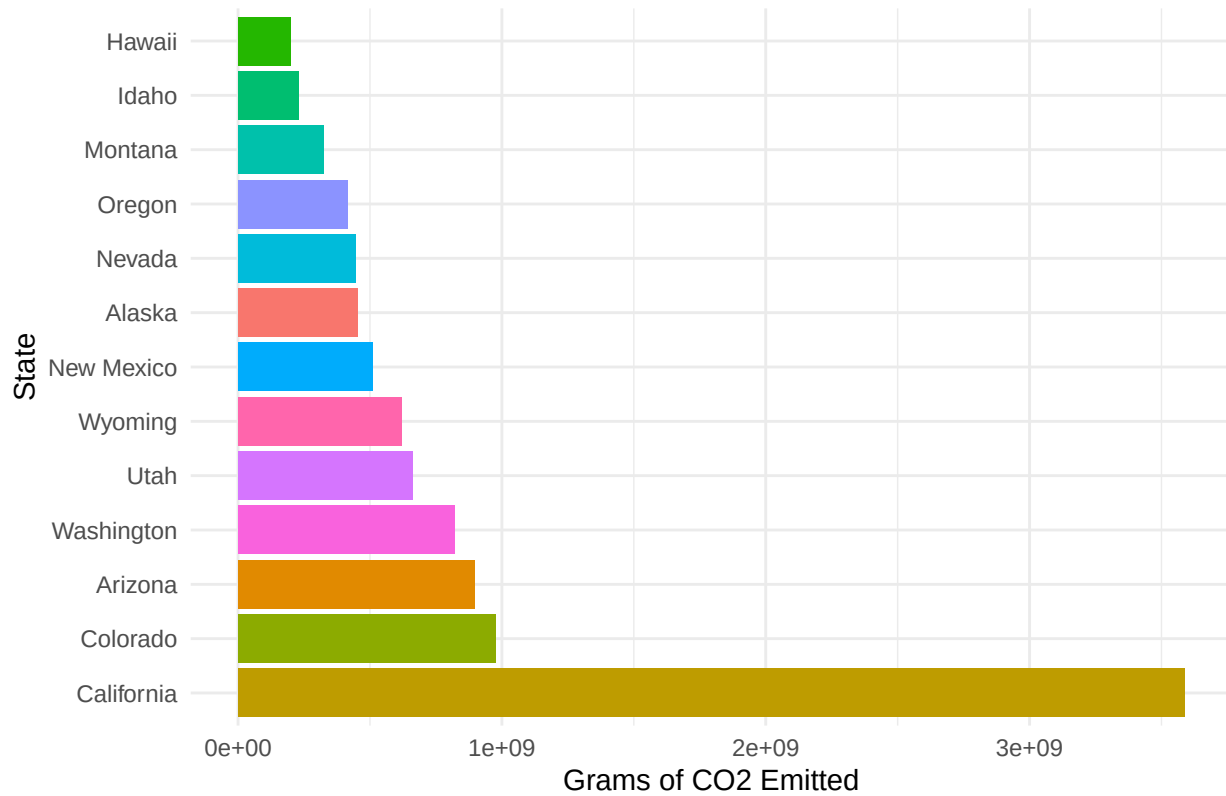
## [1] 51

# so now lets create some plots of grams of co2 emitted by each region
emissionsRegionPlot <- function(data, region_name) {
  ggplot(data, aes(x = reorder(State, -gCo2), y = gCo2, fill = State)) +
    geom_bar(stat = "identity") +
    coord_flip() + # Flip coordinates for better readability
    theme_minimal() +
    labs(title = paste(region_name, "Region: CO2 Emissions by State"),
         x = "State",
         y = "Grams of CO2 Emitted") +
    theme(legend.position = "none") # Hide legend for better visualization
}

# Plot each region
emissionsRegionPlot(westData2022, "West")

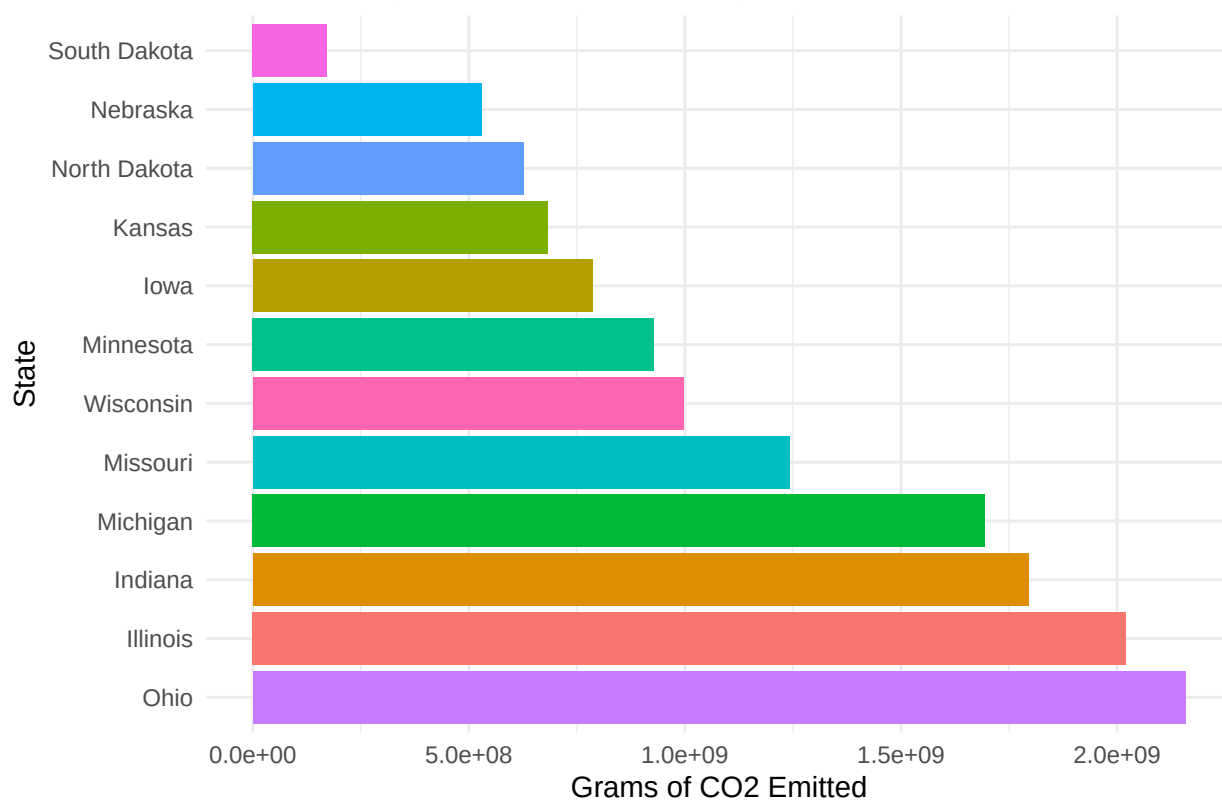
```

West Region: CO2 Emissions by State

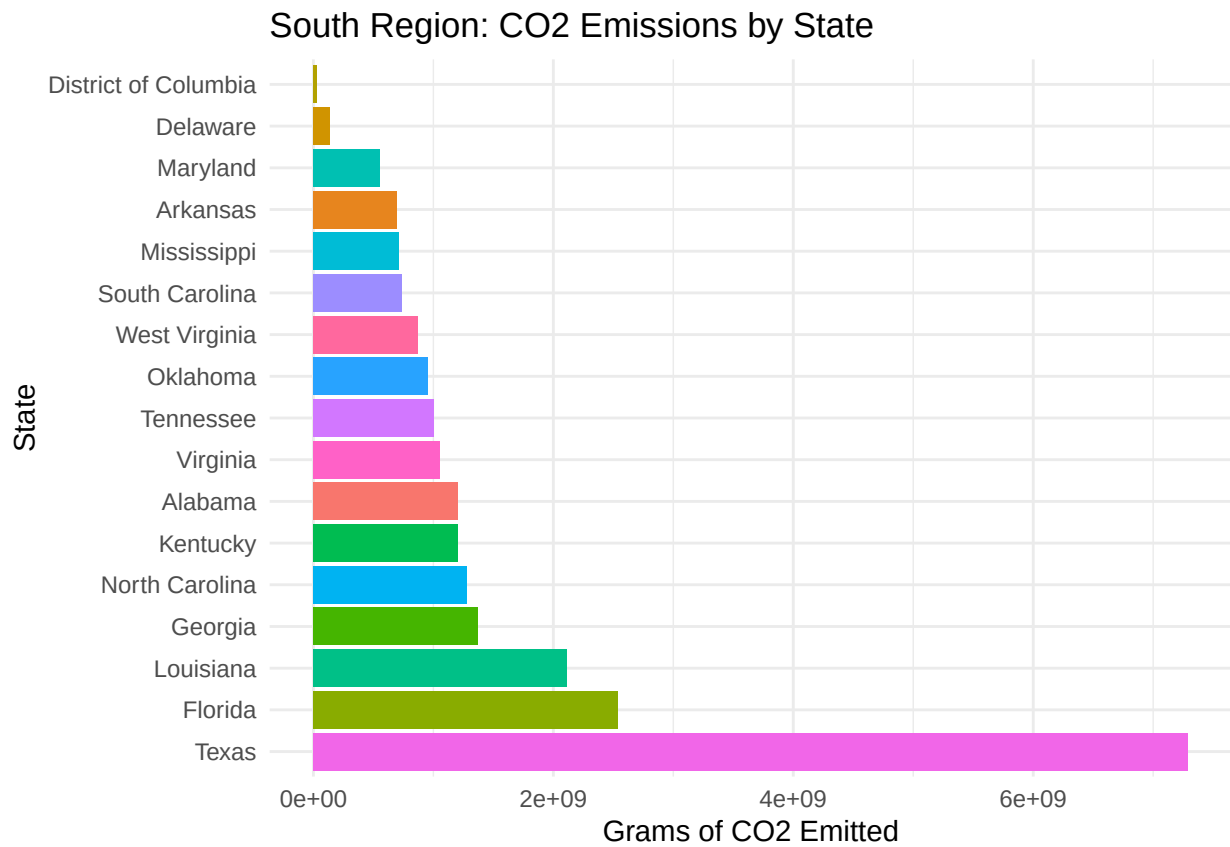


```
emissionsRegionPlot(midWestData2022, "Midwest")
```

Midwest Region: CO2 Emissions by State

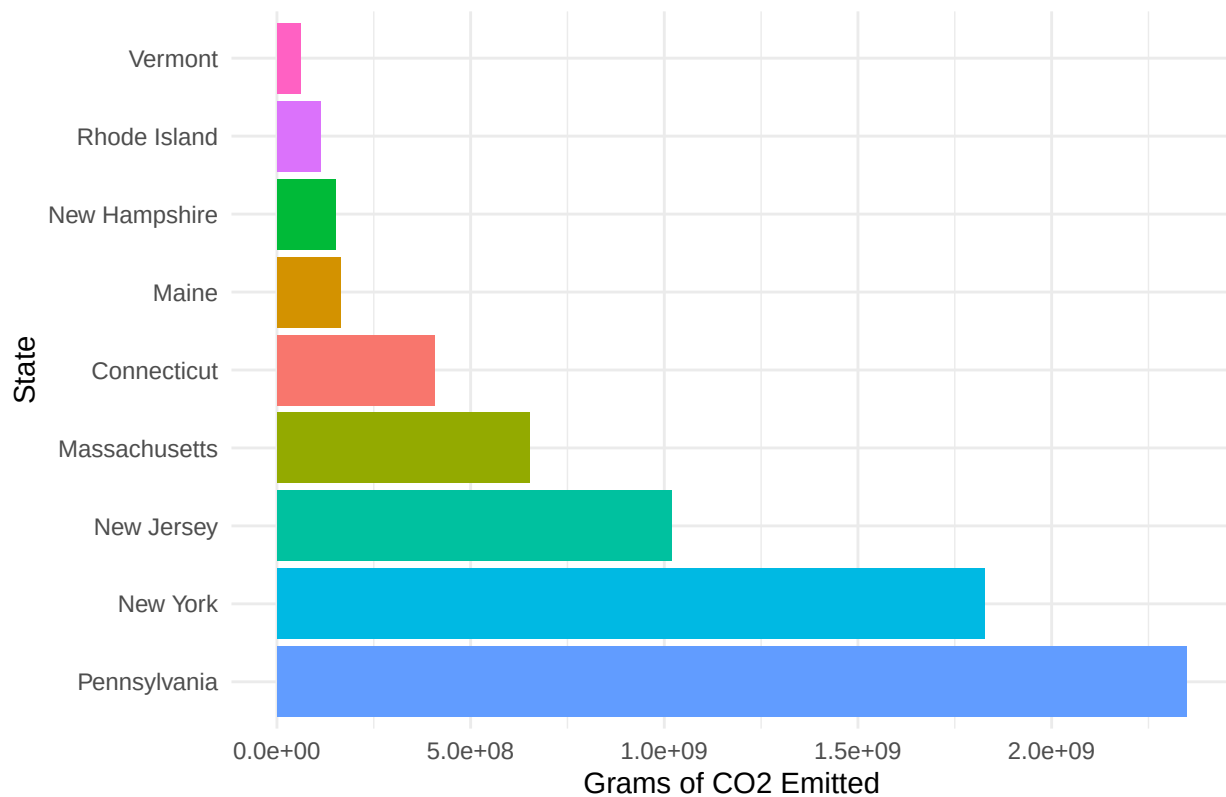


```
emissionsRegionPlot(southData2022, "South")
```

```
emissionsRegionPlot(northData2022, "North")
```

North Region: CO2 Emissions by State



```
# now I am going to do the same thing but with electricity generation
# splitting these into the same regions above
westGenerationData2022 = electricityGeneration2022 %>%
  filter(State %in% c("Arizona", "Colorado", "Idaho", "Montana", "Nevada",
    "New Mexico", "Utah", "Wyoming", "Alaska", "California",
    "Hawaii", "Oregon", "Washington"))

midWestGenerationData2022 = electricityGeneration2022 %>%
  filter(State %in% c("Illinois", "Indiana", "Michigan", "Ohio", "Wisconsin", "Iowa", "Kansas", "Minnesota",
    "Nebraska", "North Dakota", "South Dakota"))

southGenerationData2022 = electricityGeneration2022 %>%
  filter(State %in% c("Delaware", "Florida", "Georgia", "Maryland", "North Carolina", "South Carolina",
    "West Virginia", "Alabama", "Kentucky", "Mississippi", "Tennessee", "Arkansas", "Louisiana",
    "Oklahoma", "Texas", "District of Columbia"))

northGenerationData2022 = electricityGeneration2022 %>%
  filter(State %in% c("Connecticut", "Maine", "Massachusetts", "New Hampshire", "Rhode Island", "Vermont",
    "New Jersey", "New York", "Pennsylvania"))

# to ensure all data was correctly divided
sum(length(westGenerationData2022$State)) + sum(length(midWestGenerationData2022$State)) + sum(length(southGenerationData2022$State)) + sum(length(northGenerationData2022$State))

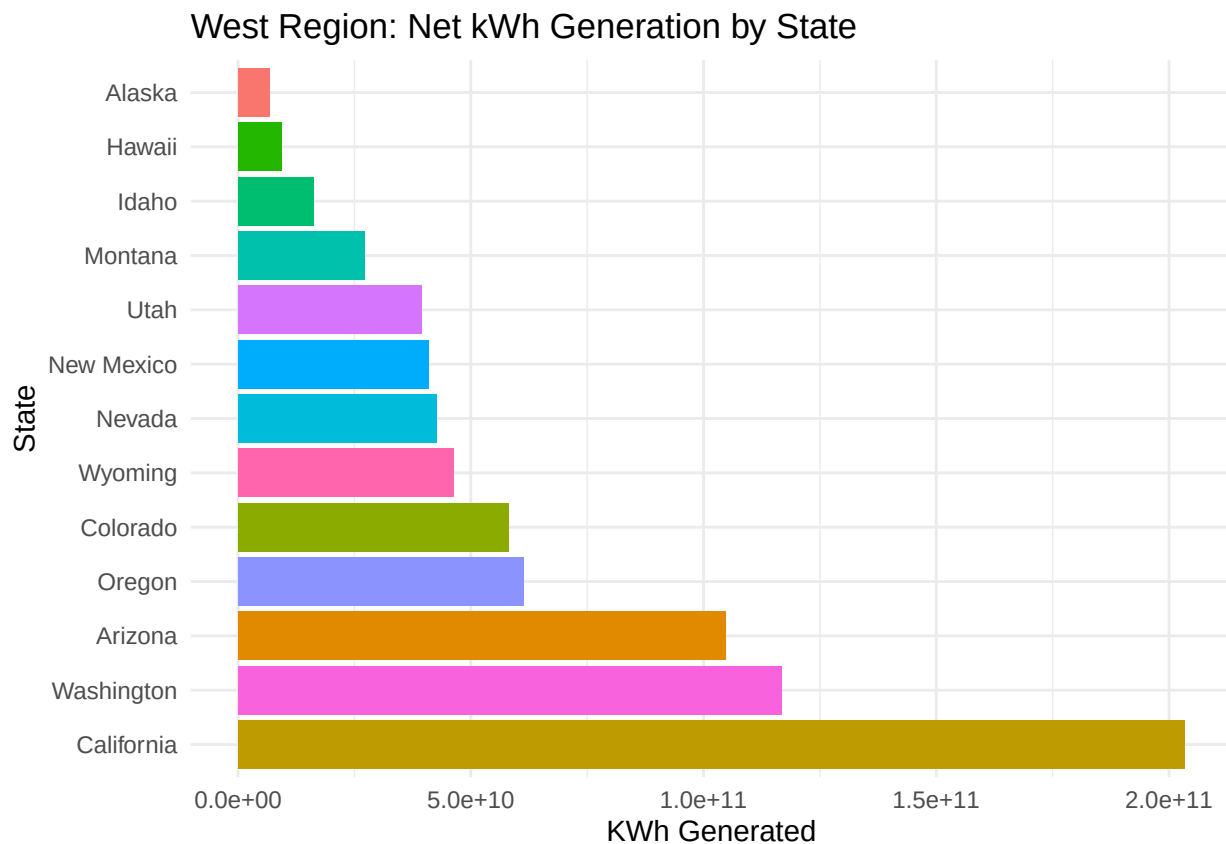
## [1] 51
```

```

# now I can create the same set of plots as above, but for electricity generation
generationRegionPlot = function(data, region_name) {
  ggplot(data, aes(x = reorder(State, -netKWhGeneration), y = netKWhGeneration, fill = State)) +
    geom_bar(stat = "identity") +
    coord_flip() + # Flip coordinates for better readability
    theme_minimal() +
    labs(title = paste(region_name, "Region: Net kWh Generation by State"),
         x = "State",
         y = "KWh Generated") +
    theme(legend.position = "none") # Hide legend for better visualization
}

# Plot each region
generationRegionPlot(westGenerationData2022, "West")

```

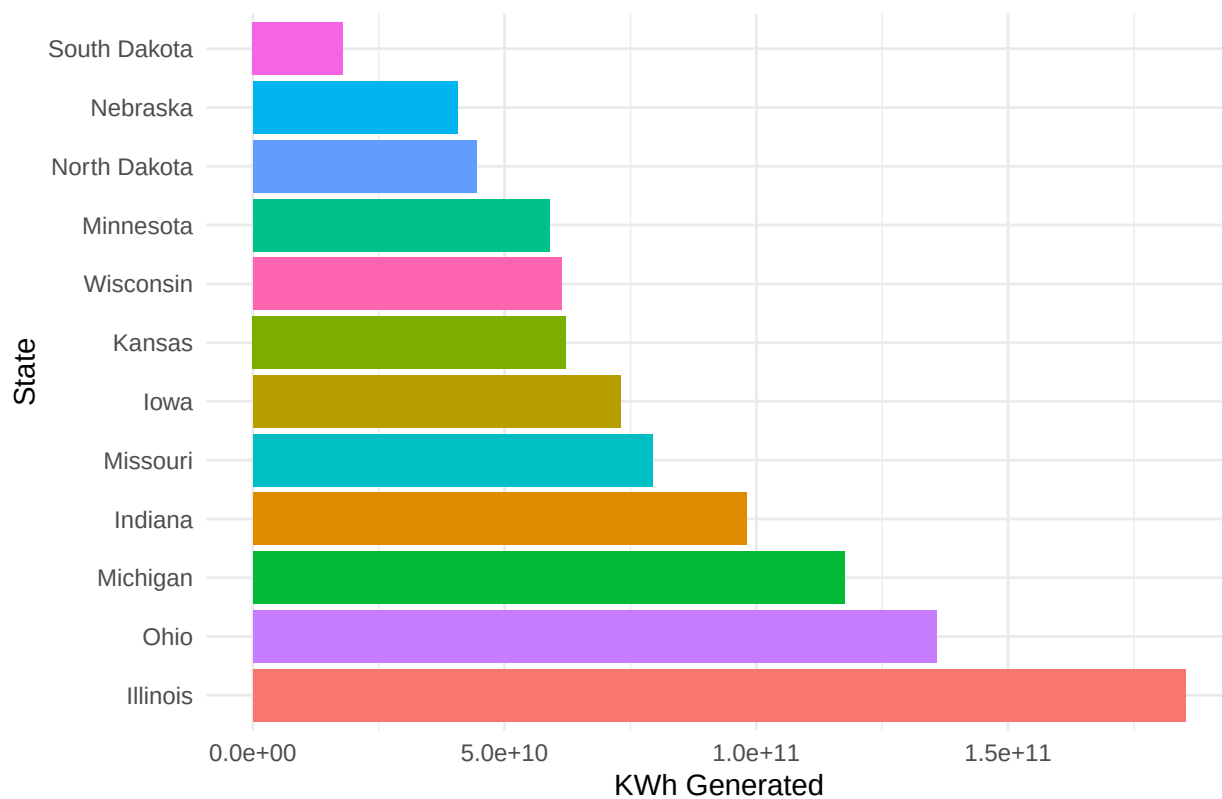


```

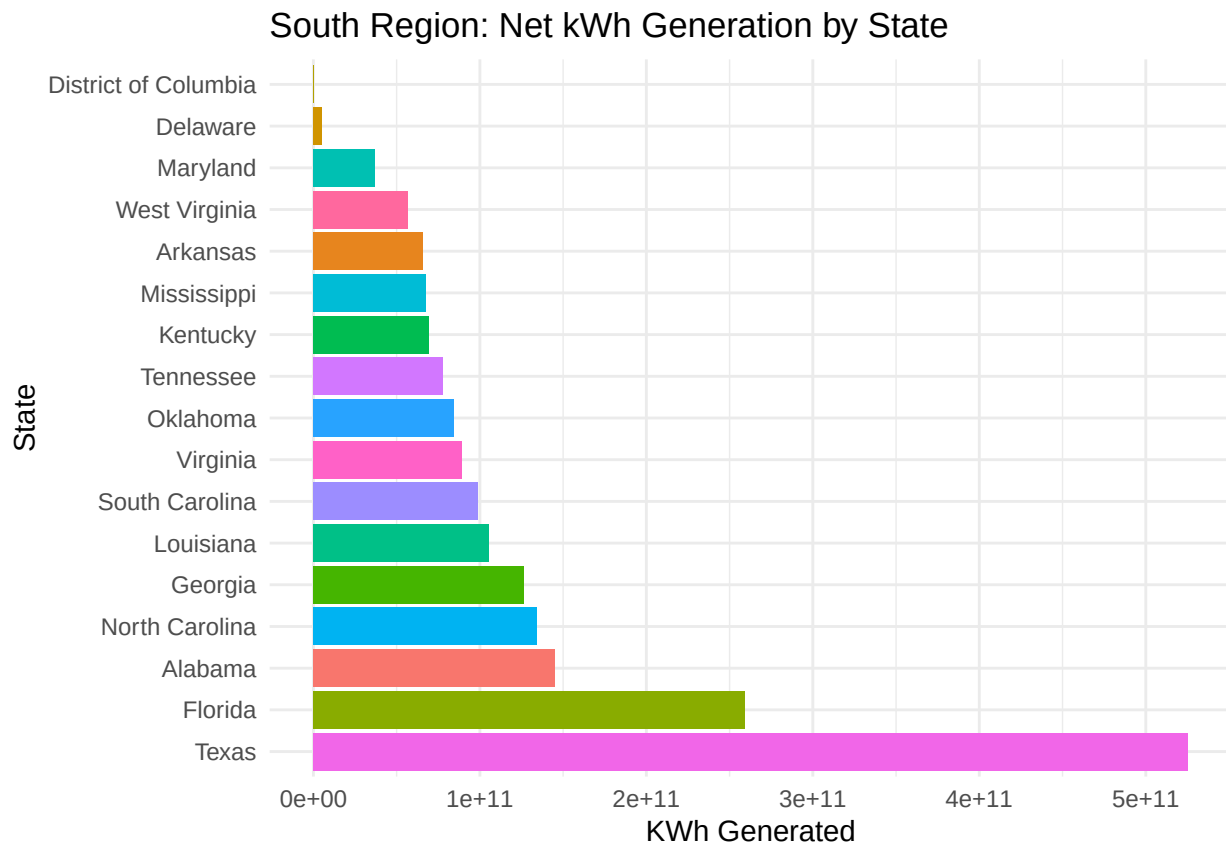
generationRegionPlot(midWestGenerationData2022, "Midwest")

```

Midwest Region: Net kWh Generation by State

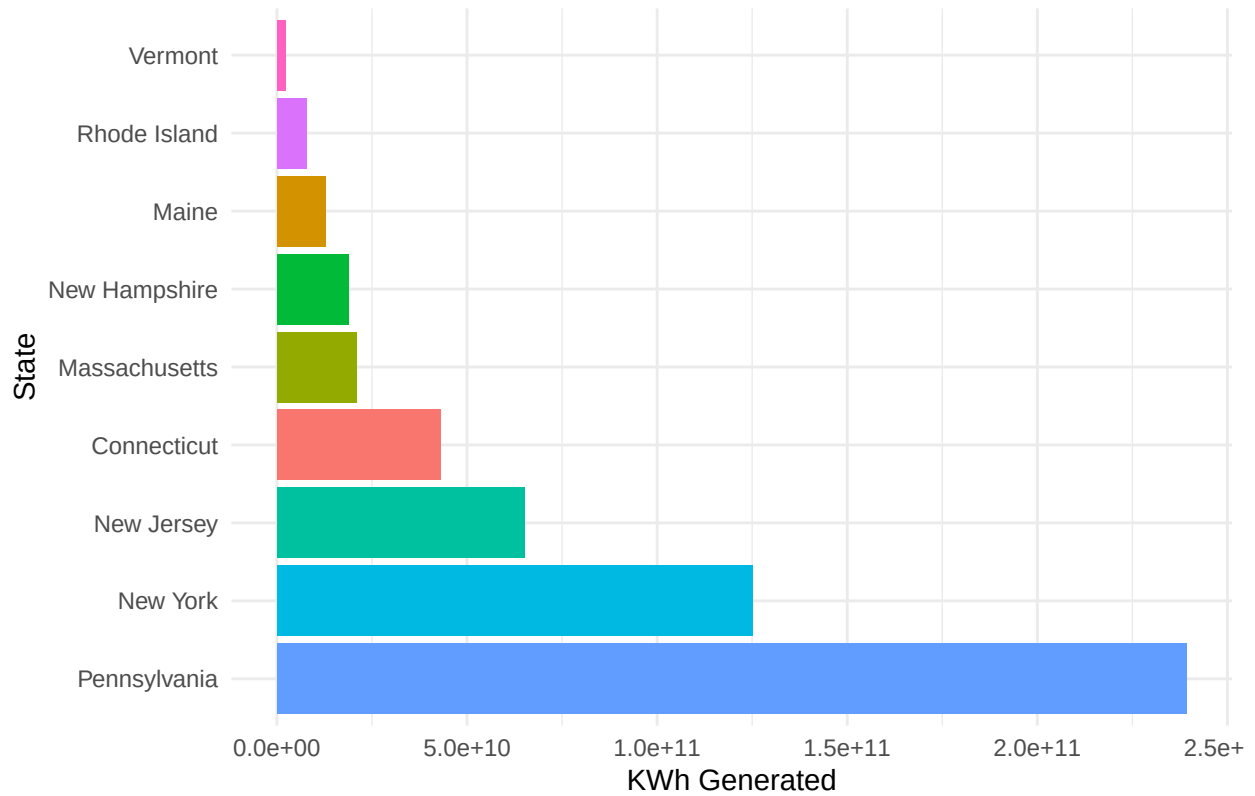


```
generationRegionPlot(southGenerationData2022, "South")
```



```
generationRegionPlot(northGenerationData2022, "North")
```

North Region: Net kWh Generation by State

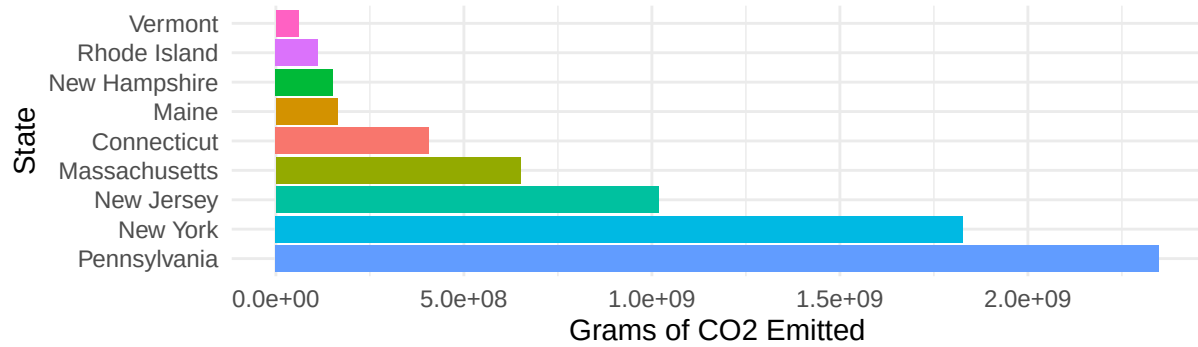


now plotting side by side by region

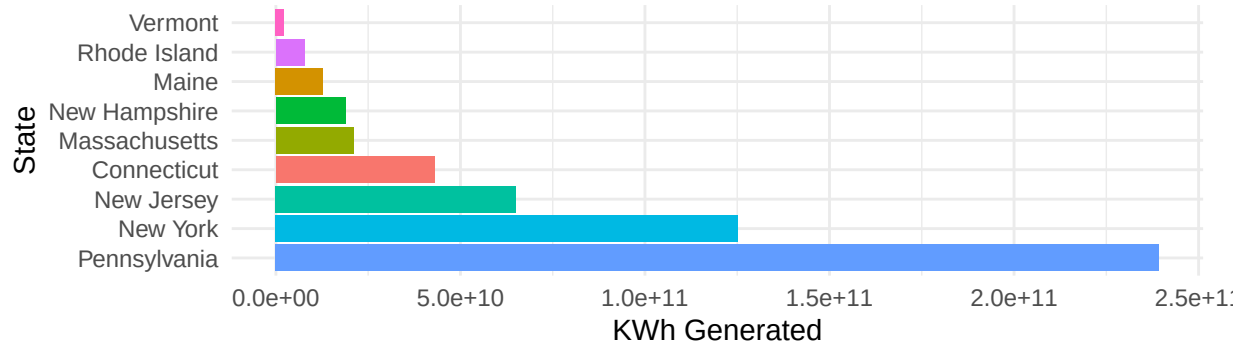
`library(patchwork)`

`emissionsRegionPlot(northData2022, "North") / generationRegionPlot(northGenerationData2022, "North")`

North Region: CO2 Emissions by State

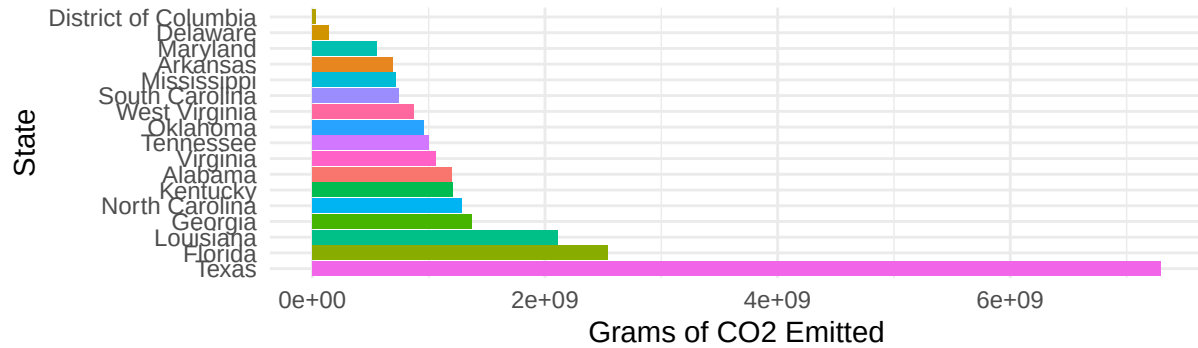


North Region: Net kWh Generation by State

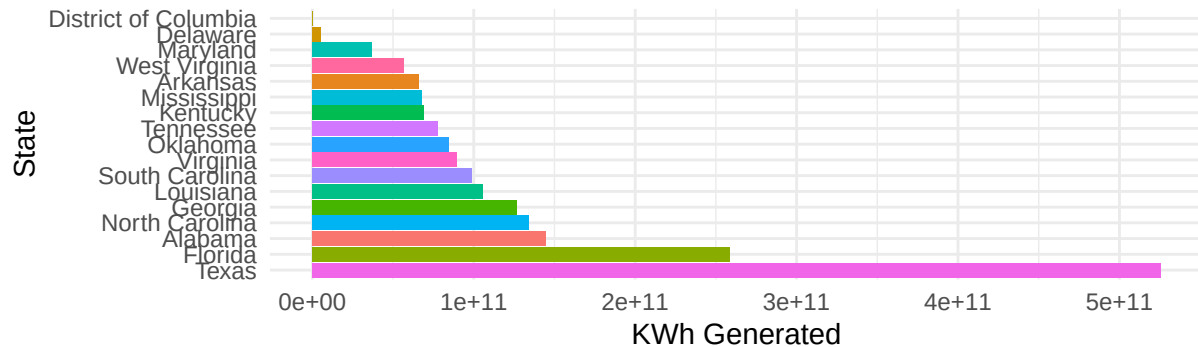


`emissionsRegionPlot(southData2022, "South") / generationRegionPlot(southGenerationData2022, "South")`

South Region: CO2 Emissions by State

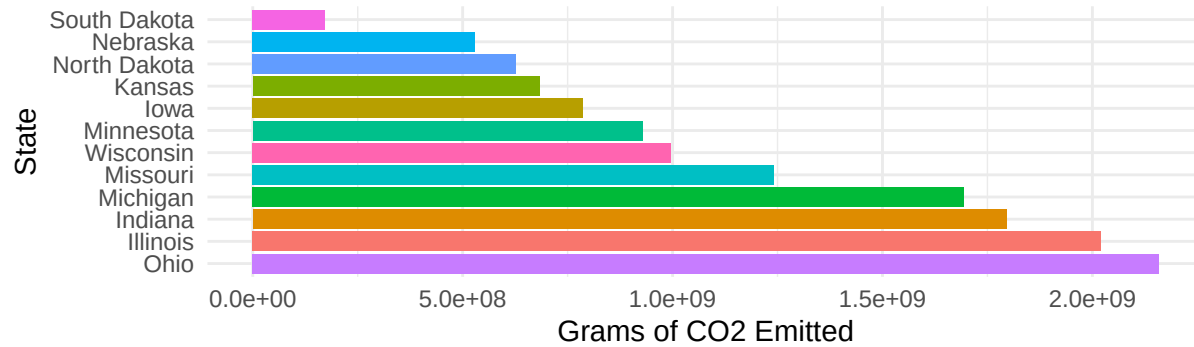


South Region: Net kWh Generation by State

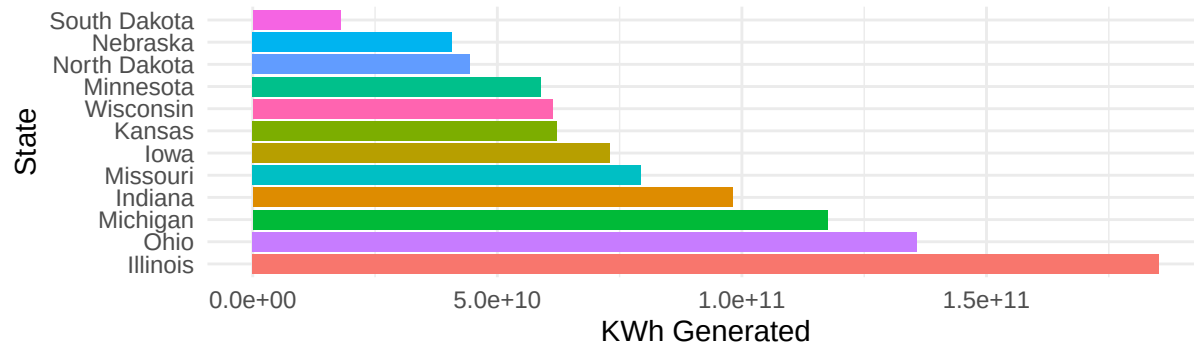


```
emissionsRegionPlot(midWestData2022, "Midwest") / generationRegionPlot(midWestGenerationData2022, "MidW
```

Midwest Region: CO2 Emissions by State

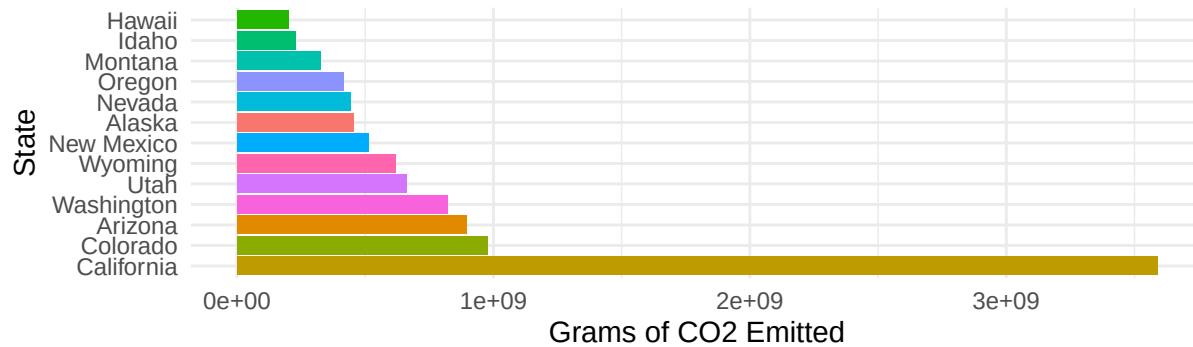


MidWest Region: Net kWh Generation by State

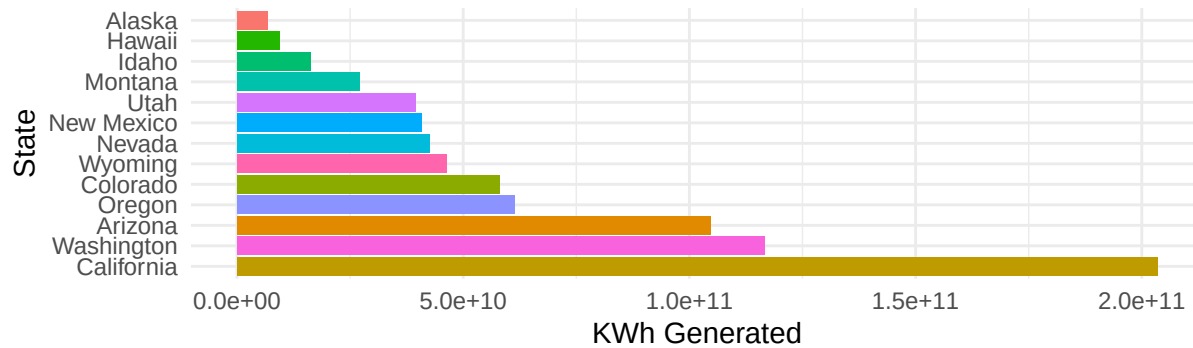


```
emissionsRegionPlot(westData2022, "West") / generationRegionPlot(westGenerationData2022, "West")
```


West Region: CO2 Emissions by State



West Region: Net kWh Generation by State



now that I have these graphs split up and visualized, I really need to join the generation and emissions data

```
finalElectricityData2022 = merge(stateData2022, electricityGeneration2022, by="State")
head(finalElectricityData2022)
```

##	State	millionTonsOfCO2	gCo2	gCo2pkWh	AverageRetailPrice.cents.kWh.
## 1	Alabama	109.3	1202300000	0	11.59
## 2	Alaska	41.3	454300000	0	20.73
## 3	Arizona	81.4	895400000	0	11.31
## 4	Arkansas	63.2	695200000	0	9.91
## 5	California	326.2	3588200000	0	22.33
## 6	Colorado	88.7	975700000	0	11.75

##	NetSummerCapacity.MW.	NetGeneration.MWh.	TotalRetailSales.MWh.
## 1	28910	144788893	87027545
## 2	2820	6694128	6002080
## 3	28202	104698773	84196517
## 4	14954	65905030	48997663
## 5	85981	203383857	251869136
## 6	18092	58044009	56763041

##	netKWhGeneration
## 1	144788893000
## 2	6694128000
## 3	104698773000
## 4	65905030000
## 5	203383857000
## 6	58044009000

```
# now we can get the value for the grams of cO2 per kilowatt hour for each state
finalElectricityData2022 = finalElectricityData2022 %>%
  mutate(gCo2pkWh = gCo2 / netKWhGeneration)
```

```
head(finalElectricityData2022)
```

```
##      State millionTonsOfCO2      gCo2      gCo2pkWh
## 1  Alabama          109.3 1202300000 0.008303814
## 2  Alaska           41.3  454300000 0.067865449
## 3  Arizona          81.4  895400000 0.008552154
## 4  Arkansas         63.2  695200000 0.010548512
## 5  California       326.2 3588200000 0.017642501
## 6  Colorado         88.7  975700000 0.016809659
##      AverageRetailPrice.cents.kWh. NetSummerCapacity.MW. NetGeneration.MWh.
## 1              11.59              28910              144788893
## 2              20.73              2820              6694128
## 3              11.31              28202             104698773
## 4               9.91              14954             65905030
## 5              22.33              85981             203383857
## 6              11.75              18092             58044009
##      TotalRetailSales.MWh. netKWhGeneration
## 1              87027545             144788893000
## 2              6002080              6694128000
## 3             84196517             104698773000
## 4             48997663             65905030000
## 5            251869136             203383857000
## 6            56763041              58044009000
```

```
# now I am going to rearrange the columns
```

```
finalElectricityData2022 = finalElectricityData2022 %>%
  select(State, millionTonsOfCO2, gCo2, gCo2pkWh, netKWhGeneration, AverageRetailPrice.cents.kWh., TotalRetailSales.MWh., NetGeneration.MWh., NetSummerCapacity.MW.)
```

```
head(finalElectricityData2022)
```

```
##      State millionTonsOfCO2      gCo2      gCo2pkWh netKWhGeneration
## 1  Alabama          109.3 1202300000 0.008303814             144788893000
## 2  Alaska           41.3  454300000 0.067865449              6694128000
## 3  Arizona          81.4  895400000 0.008552154             104698773000
## 4  Arkansas         63.2  695200000 0.010548512              65905030000
## 5  California       326.2 3588200000 0.017642501             203383857000
## 6  Colorado         88.7  975700000 0.016809659              58044009000
##      AverageRetailPrice.cents.kWh. TotalRetailSales.MWh. NetGeneration.MWh.
## 1              11.59              87027545             144788893
## 2              20.73              6002080              6694128
## 3              11.31              84196517             104698773
## 4               9.91              48997663              65905030
## 5              22.33             251869136             203383857
## 6              11.75             56763041              58044009
##      NetSummerCapacity.MW.
## 1              28910
## 2              2820
## 3             28202
```

```
## 4          14954
## 5          85981
## 6          18092
```

```
# so now I would like to add a variable of efficiency
# basically how much can you generate, with the least amount of co2
mingCo2pkWh = min(finalElectricityData2022$gCo2pkWh, na.rm = TRUE)
maxgCo2pkWh = max(finalElectricityData2022$gCo2pkWh, na.rm = TRUE)

finalElectricityData2022 = finalElectricityData2022 %>%
  mutate(EfficiencyScore = 100 * (1 - (gCo2pkWh - mingCo2pkWh) / (maxgCo2pkWh - mingCo2pkWh))) %>%
  select(State, EfficiencyScore, gCo2pkWh, netKWhGeneration, gCo2, millionTonsOfCO2, AverageRetailPrice,
         TotalRetailSales.MWh., NetSummerCapacity.MW., NetGeneration.MWh.)

head(finalElectricityData2022)
```

```
##      State EfficiencyScore   gCo2pkWh netKWhGeneration      gCo2
## 1  Alabama      99.09246 0.008303814   144788893000 1202300000
## 2  Alaska       64.41126 0.067865449     6694128000 454300000
## 3  Arizona      98.94786 0.008552154   104698773000 895400000
## 4  Arkansas     97.78543 0.010548512     65905030000 695200000
## 5 California    93.65479 0.017642501   203383857000 3588200000
## 6  Colorado     94.13973 0.016809659     58044009000 975700000
##  millionTonsOfCO2 AverageRetailPrice.cents.kWh. TotalRetailSales.MWh.
## 1              109.3                11.59              87027545
## 2              41.3                20.73              6002080
## 3              81.4                11.31              84196517
## 4              63.2                 9.91              48997663
## 5             326.2                22.33             251869136
## 6              88.7                11.75              56763041
##  NetSummerCapacity.MW. NetGeneration.MWh.
## 1              28910              144788893
## 2              2820              6694128
## 3             28202             104698773
## 4             14954             65905030
## 5             85981             203383857
## 6             18092             58044009
```

Now with this data, I can automate the efficiency based on the users input. So when the user says what state they want to see, it will fill the values for grams per mile of the electric vehicles with the formula gCo2/Kwh divided by miles/Kwh and this will show the user how electric vehicles perform in this state compared to other efficient vehicles.

```
# no we can create an efficiency scale based on the region
westEfficiencyData2022 = finalElectricityData2022 %>%
  filter(State %in% c("Arizona", "Colorado", "Idaho", "Montana", "Nevada",
                     "New Mexico", "Utah", "Wyoming", "Alaska", "California",
                     "Hawaii", "Oregon", "Washington"))

midWestEfficiencyData2022 = finalElectricityData2022 %>%
  filter(State %in% c("Illinois", "Indiana", "Michigan", "Ohio", "Wisconsin", "Iowa", "Kansas", "Minnesota",
                     "Nebraska", "North Dakota", "South Dakota"))
```

```

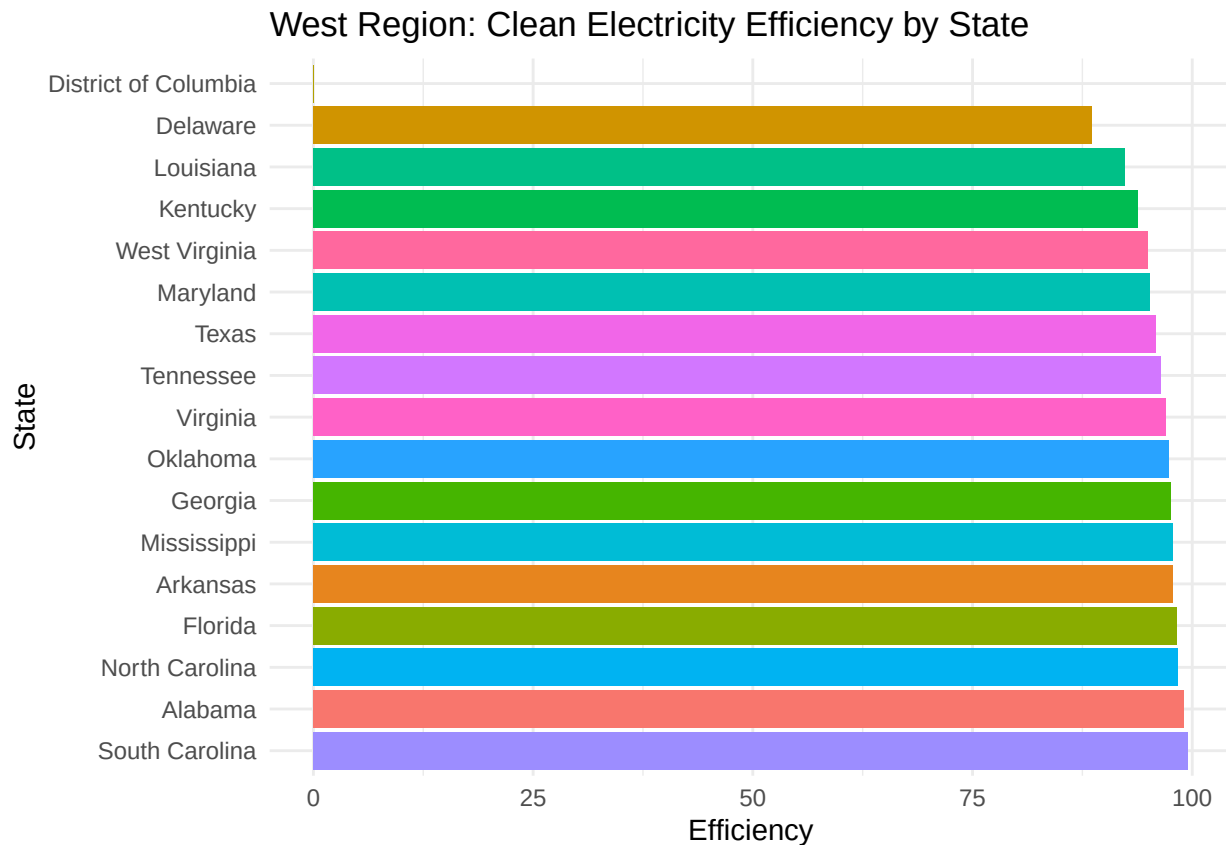
southEfficiencyData2022 = finalElectricityData2022 %>%
  filter(State %in% c("Delaware", "Florida", "Georgia", "Maryland", "North Carolina", "South Carolina",
    "West Virginia", "Alabama", "Kentucky", "Mississippi", "Tennessee", "Arkansas", "Oklahoma", "Texas", "District of Columbia"))

northEfficiencyData2022 = finalElectricityData2022 %>%
  filter(State %in% c("Connecticut", "Maine", "Massachusetts", "New Hampshire", "Rhode Island", "Vermont",
    "New Jersey", "New York", "Pennsylvania"))

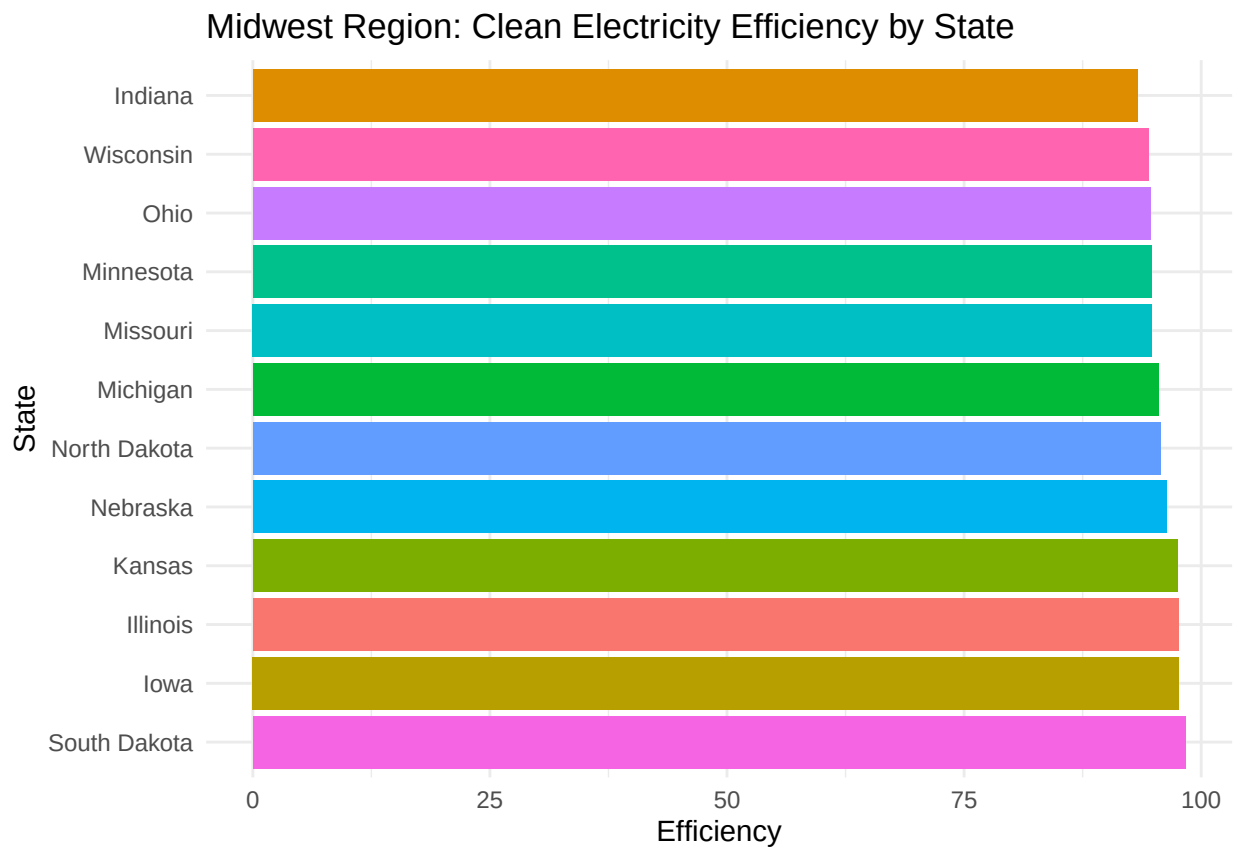
efficiencyRegionPlot = function(data, region_name) {
  ggplot(data, aes(x = reorder(State, -EfficiencyScore), y = EfficiencyScore, fill = State)) +
    geom_bar(stat = "identity") +
    coord_flip() + # Flip coordinates for better readability
    theme_minimal() +
    labs(title = paste(region_name, "Region: Clean Electricity Efficiency by State"),
      x = "State",
      y = "Efficiency") +
    theme(legend.position = "none") # Hide legend for better visualization
}

efficiencyRegionPlot(southEfficiencyData2022, "West")

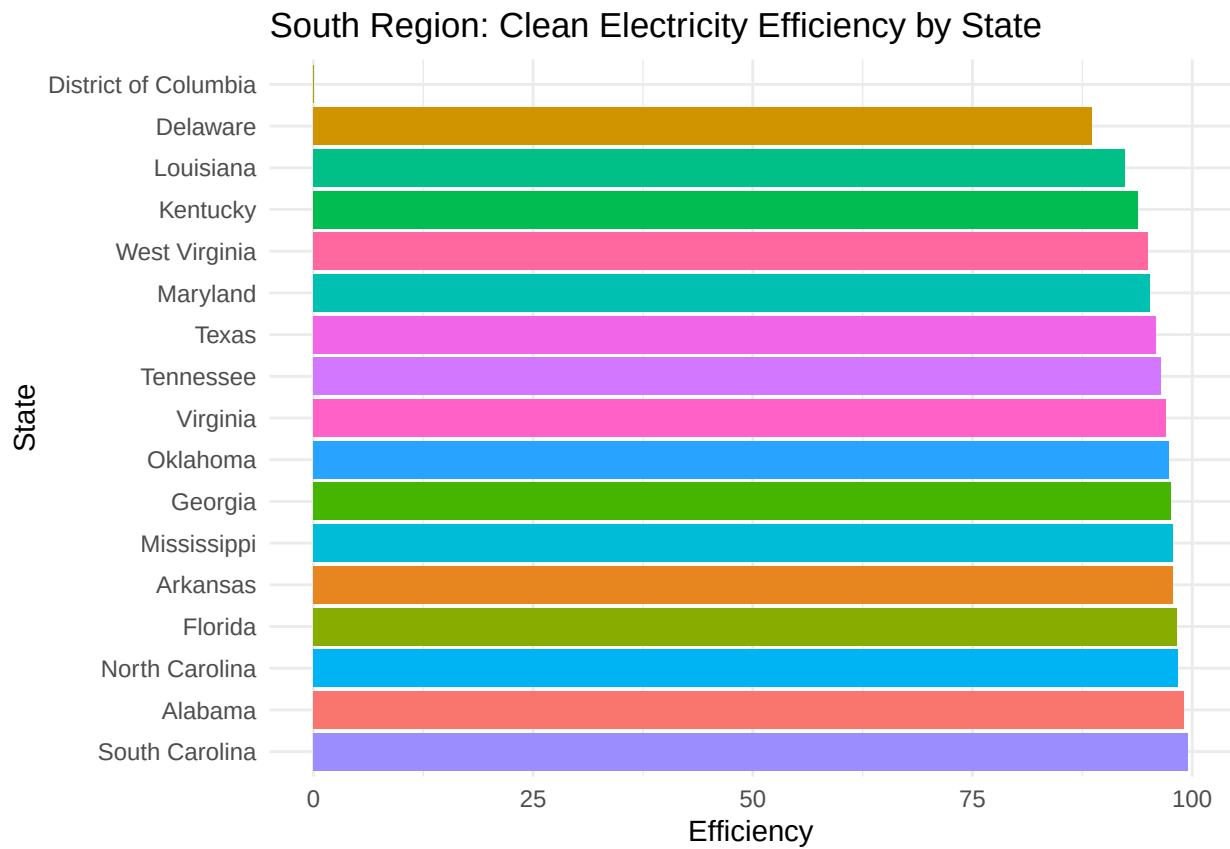
```



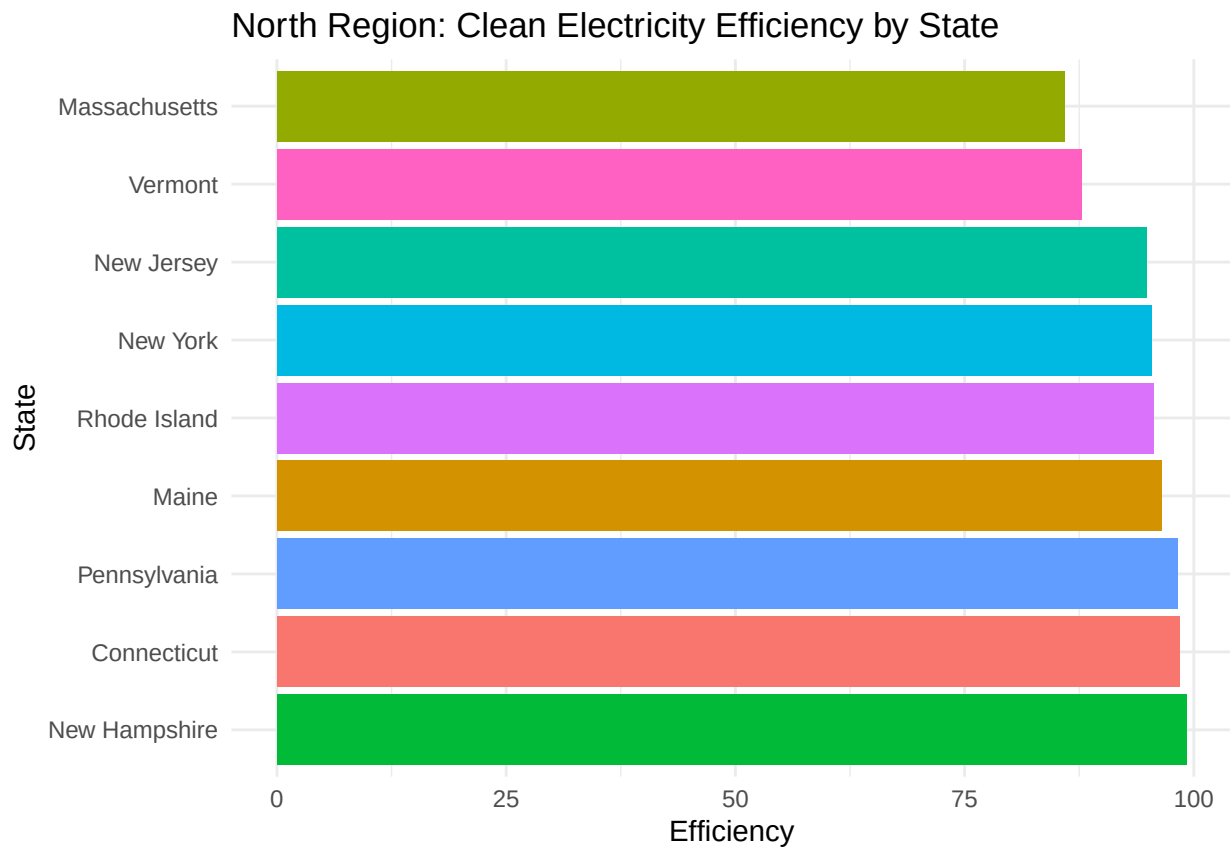
```
efficiencyRegionPlot(midWestEfficiencyData2022, "Midwest")
```



```
efficiencyRegionPlot(southEfficiencyData2022, "South")
```



```
efficiencyRegionPlot(northEfficiencyData2022, "North")
```



```
# after cleaning and manipulating the data, we will now export them as csv's to import into sql  
write_csv(nonElectricVehicleData2022, "vehicleData2022.csv")  
write_csv(stateData2022, "stateEmissionsData2022.csv")  
write_csv(electricityGeneration2022, "stateGenerationData2022.csv")  
write_csv(electricVehicles2022, "electricVehicleData2022.csv")  
write_csv(finalElectricityData2022, "completeStateData2022")
```